

WORKSHOP 2 HARI · SMK TKJ KELAS XI

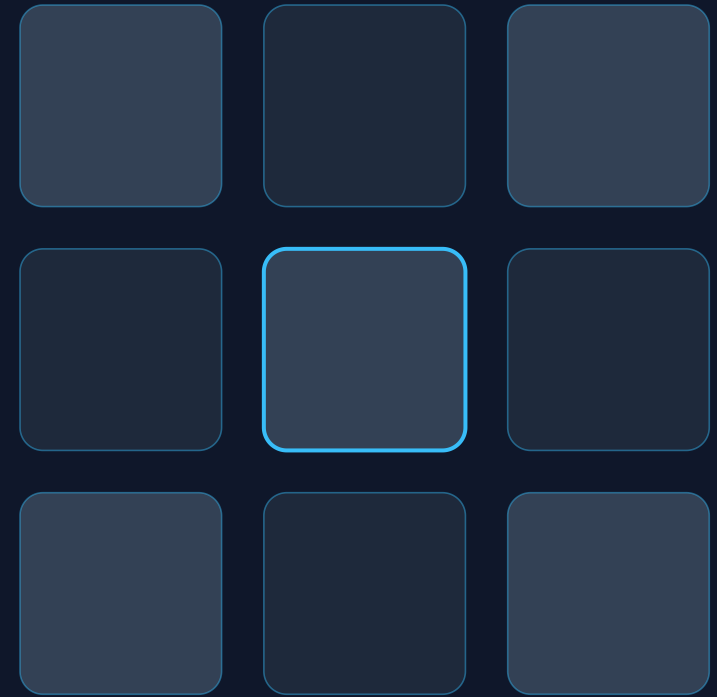
Managemen Server Cloud & Infrastruktur Virtualization (Docker)

HARI 1 — Konsep, Server, Virtualisasi, dan Docker

Disusun oleh: Cecep Saeful Azhar Hidayat, ST

www.cecepazhar.com · hi@cecepazhar.com · 0852 2069 6117

Distro acuan: Ubuntu Server LTS · Orientasi: keterampilan DevOps di dunia kerja
4 jam · Materi, demo & praktik manual di VPS masing-masing



Agenda Hari 1

00:10	Pembukaan & Praktik Terminal	Pre-test, masuk SSH ke VPS-mu, Linux CLI
00:35	Sesi 1 — Server & Cloud	On-premise vs cloud, IaaS/PaaS/SaaS, VPS
01:15	Istirahat	10 menit
01:25	Sesi 2 — Manajemen Server	SSH, scp, user, systemctl, firewall — praktik di VPS-mu
02:10	Istirahat	10 menit
02:20	Sesi 3 — Virtualisasi	Hypervisor, VM vs Container
02:55	Sesi 4 — Docker + Praktik	Konsep inti, lalu container pertama milikmu
03:50	Penutup	Recap, kuis, persiapan besok

Yang Akan Kalian Kuasai

1

Paham konsepnya

Bisa menjelaskan beda server fisik, mesin virtual, dan container dengan bahasa sendiri — bukan hafalan.

2

Bisa mengerjakannya

Menjalankan container, membuat image sendiri, dan menjalankan aplikasi web + database sekaligus.

3

Tahu memperbaikinya

Membaca pesan error, memeriksa log, dan menemukan penyebab container yang gagal jalan.

4

Siap ke dunia kerja

Mengenali bahwa ini pekerjaan DevOps sehari-hari, dan tahu langkah belajar berikutnya.

Tiga Aturan Main

Sepakati sekarang, ditegakkan sampai besok sore.

1

Salah itu wajar

Container yang error justru bahan belajar terbaik. Tidak ada yang diketawain di ruangan ini.

2

Angkat tangan saat itu juga

Kalau tertinggal, bilang sekarang — jangan tunggu sampai sesi habis. Diam = tertinggal sehabisan.

3

Semua klaim harus dibuktikan

Bilang "sudah jalan"? Tunjukkan browsernya. Ini kebiasaan paling berharga yang kalian bawa pulang.

Setiap Materi Menjawab Satu Masalah

MASALAH NYATA	SOLUSINYA	DIPELAJARI DI
"Aplikasi saya harus online 24 jam"	-> Sewa server di cloud	-> SESI 1 Cloud & VPS
"Server-nya harus diurus"	-> Kelola lewat SSH, user, firewall	-> SESI 2 Manajemen Server
"1 server 1 aplikasi itu boros"	-> Virtualisasi	-> SESI 3 VM & Hypervisor
"VM berat, boot lama, image besar"	-> Container	-> SESI 4 Docker
"Di laptop saya jalan, di server error"	-> Image yang sama di mana-mana	-> HARI 2 Praktik langsung

Tidak ada materi yang berdiri sendiri. Setiap sesi adalah jawaban dari masalah yang muncul di sesi sebelumnya.



SESI 0 · 35 MENIT

Pembukaan & Praktik Terminal

Pre-test singkat, lalu langsung masuk ke VPS milik sendiri lewat SSH dari PowerShell/CMD dan mencoba perintah Linux di sana.

Kenapa Server Pakai Teks, Bukan Klik?

1

Hemat sumber daya

Tampilan grafis memakan RAM dan CPU yang seharusnya dipakai melayani pengguna.

2

Bisa diotomasi

Perintah teks bisa ditulis jadi skrip dan dijalankan ribuan kali. Klik tidak bisa.

3

Ringan untuk remote

Cukup koneksi kecil. Tidak perlu streaming layar dari server di negara lain.

4

Bisa dicatat & diulang

Riwayat perintah adalah dokumentasi yang jujur tentang apa yang sudah dikerjakan.

Masuk ke VPS Milikmu

- 1** Buka terminal Windows
Start → ketik PowerShell → Enter. CMD juga boleh.
- 2** Masuk ke VPS-mu (akun & IP ada di kartu akun)
ssh siswa@IP-VPS
- 3** Pertama kali ditanya → ketik yes
Password tidak terlihat saat diketik, bahkan bintang pun tidak. Itu normal — ketik saja, lalu Enter.
- 4** Kenali tempatmu
whoami hostname hostname -l
- 5** Tulis nama host & IP VPS-mu di lembar kerja
Angka pertama hostname -l = IP VPS-mu (172.17.0.1 milik Docker). VPS ini milikmu sendiri selama dua hari.

Dua Dunia dalam Satu Layar

Baca tulisan di awal baris sebelum mengetik.

PC Windows

PS C:\Users\...> atau C:\...>

- Tempat kamu membuka terminal
- Perintah yang benar di sini: ssh, scp, notepad
- localhost = PC ini

VPS Linux (milikmu)

siswa@vps07:~\$

- Tempat Docker kamu berjalan
- Perintah yang benar di sini: ls, nano, sudo, docker
- localhost = VPS itu sendiri

12 Perintah Linux yang Wajib

Perintah	Artinya	Contoh
pwd	Sekarang saya ada di folder mana?	pwd
ls -la	Lihat isi folder, termasuk yang tersembunyi	ls -la /etc
cd	Pindah folder	cd /var/www
mkdir	Buat folder	mkdir proyek
cat	Tampilkan isi berkas	cat /etc/os-release
nano	Sunting berkas	nano index.html
cp / mv / rm	Salin / pindah / hapus	cp a.txt b.txt
sudo	Jalankan sebagai administrator	sudo apt update
df -h	Sisa ruang disk	df -h
free -h	Sisa RAM	free -h
ps aux	Proses apa saja yang sedang jalan	ps aux
curl	Ambil isi halaman web dari terminal	curl localhost:8080

Coba langsung di VPS-mu (Ubuntu Server LTS). Akunmu punya sudo: VPS ini milikmu sendiri, jadi salah perintah hanya merusak VPS-mu — tetap baca ulang sebelum Enter.

Coba Sekarang

```
pwd
mkdir latihan
cd latihan
nano halo.txt      tulis namamu · Ctrl+O, Enter = simpan · Ctrl+X = keluar
cat halo.txt
cd ..
ls -la
sudo whoami        minta password akunmu -> jawabannya: root

exit              keluar dari VPS
ssh siswa@IP-VPS  masuk lagi — password diminta lagi
```

Rasakan: setiap kali masuk, password diketik lagi. Catat hal-hal yang merepotkan hari ini — besok kita bahas solusinya.

Tiga Jalan Pintas yang Harus Jadi Refleks

Tab

Melengkapi nama berkas dan folder otomatis. Ini yang mencegah salah ketik — kebiasaan nomor satu yang membedakan pemula dari yang sudah terbiasa.

Coba: ketik `cd /et` lalu tekan Tab.

↑ dan ↓

Menelusuri perintah yang sudah pernah diketik. Tidak perlu menetik ulang perintah panjang.

Ctrl + C

Menghentikan perintah yang sedang berjalan. Wajib tahu sebelum menjalankan apa pun.

Alamat IP dan Port

- Alamat IP = nomor rumah sebuah komputer di jaringan
- 127.0.0.1 atau localhost = komputer ini sendiri
- 192.168.x.x = alamat lokal di jaringan sekolah atau rumah
- Port = nomor pintu di rumah tersebut. Satu komputer punya 65.535 pintu
- Penulisannya: alamat, titik dua, nomor pintu → localhost:8080

Analogi

Alamat IP adalah alamat rumah.

Port adalah nomor pintu di rumah itu.

Kalau kamu kirim paket ke "Jl. Merdeka No. 10" tapi tidak bilang pintu mana, paketnya bingung.

Itulah kenapa kita menulis
localhost:8080

Port yang Akan Sering Muncul

Port	Layanan	Ingat sebagai
22	SSH	Pintu belakang khusus admin
80	HTTP	Pintu depan web, tanpa gembok
443	HTTPS	Pintu depan web, bergembok
3306	MySQL / MariaDB	Gudang data
5432	PostgreSQL	Gudang data
8080	HTTP alternatif	Pintu cadangan — paling sering kita pakai di workshop ini

Port 22, 80, dan 443 wajib hafal. Tiga ini muncul di hampir setiap pekerjaan server.

Model Client — Server

```
CLIENT                SERVER
(browser)             (aplikasi web)
|                     |
| --- 1. "Tolong kirim halaman /index" --> |
|                     | 2. Diproses
| <-- 3. "Ini halamannya" (HTML) ----- |
|                     |
Menampilkan           Menyala terus 24 jam
```

Semua yang dipelajari dua hari ini pada dasarnya adalah cara menyiapkan dan merawat sisi kanan diagram ini.



SESI 1 · 40 MENIT

Server & Cloud Computing

Apa itu server, kenapa orang menyewa daripada membeli, dan bagaimana membaca halaman penjualan VPS tanpa bingung.

Komputer Pribadi vs Server

Server bukan jenis komputer khusus. Yang membedakan adalah perannya, bukan bentuknya.

Aspek	Komputer pribadi	Server
Waktu nyala	Beberapa jam sehari	24 jam × 365 hari
Prioritas spesifikasi	Grafis, layar bagus	RAM besar, disk & jaringan cepat
Tampilan grafis	Wajib ada	Umumnya tidak ada
Kalau mati	Pemiliknya kesal	Ratusan pengguna terganggu
Komponen cadangan	Jarang	PSU ganda, disk RAID, jaringan ganda

Cek paham: kalau laptop kalian dipakai jadi server web, apa kelemahan terbesarnya?

On-Premise vs Cloud

On-Premise

Beli dan taruh di lokasi sendiri

- Kendali penuh atas perangkat keras dan data
- Tidak bergantung internet untuk akses lokal
- Modal awal besar dan keluar di muka
- Menambah kapasitas butuh berminggu-minggu
- Kalau rusak, biaya ditanggung sendiri
- Mudah membeli kapasitas berlebih yang menganggur

Cloud

Sewa dari penyedia, bayar per bulan/jam

- Tidak perlu modal besar di awal
- Kapasitas bertambah dalam hitungan menit
- Perawatan perangkat keras jadi urusan penyedia
- Biaya berulang seumur layanan dipakai
- Wajib ada internet, tidak ada kendali fisik
- Tagihan bisa membengkak kalau tidak dipantau

ANALOGI

On-premise itu beli motor.
Cloud itu naik ojek online.

Beli motor: modal besar di depan, servis dan pajak tanggung sendiri — tapi motornya milikmu.

Ojek online: bayar per pakai, tidak pusing servis, bisa ganti ke mobil kalau tiba-tiba rombongan.

Tapi kalau dipakai tiap hari sepanjang tahun, total ojol bisa lebih mahal daripada beli motor.

Tanyakan ke kelas: "Kalau kalian butuh kendaraan cuma 3 kali sebulan, pilih mana?"

IaaS, PaaS, SaaS

Materi yang paling sering keluar di ujian, dan paling sering tertukar. Satu analogi saja: pizza.



IaaS

Infrastructure as a Service

Diberi mesin kosong: CPU, RAM, disk.
OS ke atas urusan kamu.

AWS EC2, DigitalOcean, Biznet Gio,
VPS



PaaS

Platform as a Service

Diberi lingkungan siap pakai. Tinggal
unggah kode program.

Heroku, Vercel, Railway



SaaS

Software as a Service

Perangkat lunak jadi, langsung
dipakai lewat browser.

Gmail, Google Docs, Canva, Zoom

Siapa Mengurus Apa

Model	Analoginya	Kamu urus	Penyedia urus
On-Premise	Bikin pizza dari nol di rumah	Dapur, oven, bahan, adonan, masak	—
IaaS	Beli bahan, masak di rumah	Resep, topping, memanggang, penyajian	Adonan & bahan mentah
PaaS	Pizza beku, tinggal panggang	Memanggang & penyajian	Adonan, saus, topping
SaaS	Pesan pizza jadi, diantar	Makan saja	Semuanya

Makin ke bawah: makin sedikit yang kamu urus, tapi makin sedikit pula yang bisa kamu atur.

Pembagian Tanggung Jawab

LAPISAN	On-Prem	IaaS	PaaS	SaaS

Aplikasi	KAMU	KAMU	KAMU	penyedia
Data	KAMU	KAMU	KAMU	KAMU
Runtime	KAMU	KAMU	penyedia	penyedia
Middleware	KAMU	KAMU	penyedia	penyedia
Sistem Operasi	KAMU	KAMU	penyedia	penyedia
Virtualisasi	KAMU	penyedia	penyedia	penyedia
Server fisik	KAMU	penyedia	penyedia	penyedia
Penyimpanan	KAMU	penyedia	penyedia	penyedia
Jaringan	KAMU	penyedia	penyedia	penyedia

Perhatikan baris DATA: pada keempat model, data tetap tanggung jawab kamu. "Pindah ke cloud" tidak pernah berarti keamanan data jadi urusan orang lain.

Tebak Modelnya

Lempar cepat ke kelas, satu per satu. Jangan beri jawaban sebelum ada yang mencoba.



Instagram

bagi penggunanya

Jawaban: SaaS



VPS berisi
Ubuntu kosong

Jawaban: IaaS



Vercel untuk
situs Next.js

Jawaban: PaaS



Google Drive

Jawaban: SaaS

Membaca Halaman Penjualan VPS

VPS adalah bentuk IaaS paling terjangkau — dan yang paling sering dipakai siswa maupun freelancer.

Spesifikasi	Artinya	Pengaruhnya
vCPU	Jumlah inti prosesor virtual	Kecepatan memproses permintaan
RAM	Memori kerja	Berapa aplikasi bisa jalan bersamaan
Storage SSD/NVMe	Ruang penyimpanan	Kapasitas berkas; NVMe jauh lebih cepat
Bandwidth	Kuota data keluar-masuk per bulan	Batas trafik sebelum kena biaya tambahan
Uptime SLA	Jaminan waktu nyala	99,9% berarti boleh mati ± 43 menit/bulan
Region	Letak fisik data center	Makin dekat pengguna, makin rendah latensi

Satu Angka di Belakang Koma, Beda 10 Kali Lipat

Berapa lama sebuah layanan boleh mati dalam satu bulan (30 hari):

72 jam

SLA 90%
Hampir tidak ada layanan serius
memakai ini

43 menit

SLA 99,9%
Standar umum VPS terjangkau

4,3 menit

SLA 99,99%
Harganya jauh lebih mahal

Tanyakan ke kelas: untuk website sekolah, 99,9% cukup tidak? Untuk aplikasi bank? — Inilah cara seorang DevOps memilih layanan: bukan "yang paling bagus", tapi "yang cukup untuk kebutuhan ini".

Memilih Ukuran & Penyedia

- Belajar, 1 website statis → 1 vCPU / 1 GB RAM
- Website + database, trafik kecil → 1-2 vCPU / 2 GB RAM
- Beberapa container Docker → 2 vCPU / 4 GB RAM
- Aplikasi produksi kecil + pemantauan → 4 vCPU / 8 GB RAM
- Server di dalam negeri = latensi lebih rendah untuk pengguna Indonesia

Penyedia

INDONESIA

Biznet Gio · IDCloudHost
Niagahoster · Rumahweb
Dewaweb

GLOBAL

AWS · Google Cloud · Azure
DigitalOcean · Vultr
Linode · Hetzner

Ukur VPS Kalian Sendiri

VPS yang kalian masuki di Sesi 0 adalah VPS sungguhan: potongan dari satu server fisik sekolah.

`nproc` jumlah vCPU

`free -h` RAM

`df -h /` disk

VPS kalian masuk baris mana di tabel ukuran tadi? Cukup untuk apa saja?

Empat Kasus, Empat Kelompok

Tiap kelompok menyampaikan jawaban maksimal 30 detik. Yang dinilai alasannya, bukan pilihannya.

A

Warung kopi

Ingin website menu sederhana yang jarang berubah.

Model apa? Spesifikasi apa?

B

Sekolah

Ingin sistem absensi sendiri yang menyimpan data siswa.

Di mana sebaiknya ditaruh?

C

Tim lomba

Butuh server untuk menguji aplikasi selama 3 hari lomba saja.

Bagaimana caranya hemat?

D

Toko online

Ramai hanya saat tanggal kembar, sepi di hari biasa.

Apa yang harus dipilih?

2

SESI 2 · 45 MENIT · PRAKTIK DI VPS-MU

Manajemen Server Dasar

SSH, kirim berkas, pengguna, layanan, paket, firewall, pemantauan, dan hardening — dicoba sendiri di VPS masing-masing. Hanya kunci SSH dan hardening yang didemokan.

Menghubungi Server dari Jauh

SSH adalah cara standar masuk ke server. Semua lalu lintasnya terenkripsi.

```
ssh namauser@alamat_server
```

```
ssh siswa@103.145.22.10
```

```
ssh -p 2222 siswa@103.145.22.10 # kalau port SSH diubah
```

Saat koneksi pertama, server memperkenalkan diri:

```
The authenticity of host '103.145.22.10' can't be established.
```

```
ED25519 key fingerprint is SHA256:xxxx...
```

```
Are you sure you want to continue connecting (yes/no)?
```

Kalau sidik jari ini berubah tanpa alasan, bisa jadi kamu sedang dikelabui dan menyambung ke server palsu. Jangan asal ketik yes.

Password vs Kunci SSH

Autentikasi Password

Untuk belajar saja

- Ketik sandi setiap kali masuk
- Bisa ditebak paksa (brute force)
- Harus diingat dan diketik
- Kalau bocor, siapa pun bisa masuk

Autentikasi Kunci SSH

Wajib untuk server sungguhan

- Dua berkas: kunci privat & kunci publik
- Praktis mustahil ditebak
- Otomatis, tanpa mengetik apa pun
- Kunci privat tidak pernah dikirim ke mana pun
- Bisa diberi passphrase sebagai lapis kedua

Membuat & Memasang Kunci SSH

```
ssh-keygen -t ed25519 -C "siswa@smk"
```

Menghasilkan dua berkas di ~/.ssh/ :

id_ed25519 <- KUNCI PRIVAT. Rahasia.
Tidak boleh dikirim ke siapa pun,
termasuk ke guru.

id_ed25519.pub <- Kunci publik. Ini yang ditaruh
di server.

```
ssh-copy-id siswa@103.145.22.10
```

Setelah itu: `ssh siswa@103.145.22.10` -> langsung masuk

Kunci tidak dibuat di PC lab: PC dipakai bergantian. Di dunia kerja, login password ke server produksi biasanya dimatikan total.

Mengirim Berkas ke VPS: scp

- 1** Buka PowerShell KEDUA
Jendela pertama biarkan tetap masuk ke VPS.
- 2** Buat berkas di PC
notepad catatan.txt → Yes → tulis nama & kelas → simpan
- 3** Kirim ke VPS — diketik di PC, bukan di VPS
scp catatan.txt siswa@IP-VPS:~/
- 4** Buktikan di jendela SSH
ls cat catatan.txt

root vs Pengguna Biasa

- root punya kuasa penuh — bisa menghapus apa pun, termasuk sistem operasinya sendiri
- Aturan emas: jangan bekerja sehari-hari sebagai root
- Buat user biasa, lalu beri hak sudo saat memang butuh
- Kalau akun user biasa dibobol, dampaknya jauh lebih terbatas

Perintahnya (Ubuntu)

```
sudo adduser siswa
```

```
sudo usermod -aG sudo siswa
```

```
su - siswa
```

```
# cek keanggotaan grup  
groups siswa
```

Membaca Izin Berkas

```
$ ls -la
-rw-r--r-- 1 siswa siswa 1024 Sep 14 10:00 catatan.txt
drwxr-xr-x 2 siswa siswa 4096 Sep 14 10:00 folder/

-   rw-   r--   r--
|   |   |   |
tipe pemilik grup publik

tipe : - berkas biasa   d folder   l tautan
izin : r baca (4)  w tulis (2)  x jalankan (1)

rwx = 4+2+1 = 7    r-x = 4+0+1 = 5    r-- = 4

chmod 755 skrip.sh    # pemilik penuh, lain baca & jalankan
chmod 600 rahasia.txt # hanya pemilik yang boleh baca & tulis
```

Kenapa kunci privat SSH harus chmod 600? Supaya pengguna lain di server yang sama tidak bisa membacanya — SSH bahkan menolak jalan kalau izinnya terlalu longgar.

Coba: Buat Akun Kedua

```
sudo adduser tamu          isi password tamu 2x, sisanya Enter
echo "rahasia siswa" > /tmp/rahasia.txt
ls -l /tmp/rahasia.txt     -rw-rw-r--
sudo -u tamu cat /tmp/rahasia.txt -> terbaca

chmod 600 /tmp/rahasia.txt
ls -l /tmp/rahasia.txt     -rw-----
sudo -u tamu cat /tmp/rahasia.txt -> Permission denied

sudo deluser --remove-home tamu
rm /tmp/rahasia.txt
```

Setelah chmod 600 hanya pemiliknya yang boleh membaca — tamu ditolak. Berkasnya di /tmp karena folder home di Ubuntu sudah tertutup untuk akun lain.

Mengelola Layanan dengan systemctl

Perintah	Fungsi
systemctl status nginx	Lihat kondisi — jalankan ini pertama saat ada masalah
systemctl start nginx	Nyalakan sekarang
systemctl stop nginx	Matikan
systemctl restart nginx	Matikan lalu nyalakan
systemctl reload nginx	Muat ulang konfigurasi tanpa memutus koneksi
systemctl enable nginx	Nyala otomatis saat server dihidupkan
journalctl -u nginx -n 50	Lihat 50 baris log terakhir layanan tersebut

start $\neq$ enable

start → menyalakan SEKARANG

enable → menyalakan SETIAP KALI server dinyalakan ulang

- Banyak pemula lupa menjalankan enable, lalu bingung kenapa layanannya mati setelah server direstart
- Di server produksi, layanan yang tidak enable berarti: server reboot jam 3 pagi, aplikasi tidak hidup lagi, dan baru ketahuan pagi harinya
- Kebiasaan yang aman: setiap kali menjalankan start, langsung jalankan enable juga
- Cek dengan `systemctl is-enabled nginx` — jawabannya harus "enabled"

UFW — Menentukan Pintu Mana yang Boleh Dibuka

Server tanpa firewall seperti rumah dengan semua jendela terbuka.

```
sudo ufw status verbose
```

```
sudo ufw allow 22/tcp      # SSH <-- LAKUKAN INI DULUAN
```

```
sudo ufw allow 80/tcp      # HTTP
```

```
sudo ufw allow 443/tcp     # HTTPS
```

```
sudo ufw show added       # 22/tcp HARUS ada di daftar
```

```
sudo ufw enable
```

```
sudo ufw status numbered
```

```
sudo ufw delete 3         # hapus aturan nomor 3
```

Jawab y di ufw enable HANYA kalau 22/tcp tampil di ufw show added. Lalu buka PowerShell baru dan masuk lagi. Terkunci? Angkat tangan — admin menolong lewat konsol hypervisor.

KISAH NYATA YANG MASIH TERJADI TIAP HARI

Dia menyalakan firewall sebelum mengizinkan port 22.

Firewall langsung memutus koneksi SSH-nya sendiri. Servernya di cloud, tidak ada akses fisik.
Dia terkunci di luar servernya sendiri.

Satu-satunya jalan: konsol darurat dari panel penyedia — atau membangun ulang server dari nol.

Pelajarannya: izinkan SSH dulu, baru nyalakan firewall.

Pemantauan Server

Perintah	Menjawab pertanyaan
uptime	Sudah berapa lama menyala, seberapa sibuk?
free -h	Sisa RAM berapa?
df -h	Sisa disk berapa?
top / htop	Apa yang sedang memakan CPU dan RAM?
du -sh *	Folder mana yang paling besar?
ss -tulpn	Port mana yang terbuka & program apa yang memakainya?
journalctl -xe	Apa error terakhir di sistem?

Coba berurutan: uptime → free -h → df -h → top (q untuk keluar) → sudo ss -tulpn → who. Urutan pertama itu menjawab 80% kasus server lemot; penyebab tersering justru disk penuh karena log menumpuk.

Enam Langkah pada Setiap Server Baru

Dikerjakan sebelum server dipakai — bukan sesudah ada masalah.

- 1** **Buat pengguna non-root**
`sudo adduser siswa && sudo usermod -aG sudo siswa`
- 2** **Aktifkan kunci SSH, matikan login password**
`PasswordAuthentication no` di `/etc/ssh/sshd_config`
- 3** **Matikan login root lewat SSH**
`PermitRootLogin no` di `/etc/ssh/sshd_config`
- 4** **Nyalakan firewall, buka hanya port yang dipakai**
`sudo ufw allow 22/tcp && sudo ufw enable`
- 5** **Pasang fail2ban**
`sudo apt install fail2ban` — memblokir IP yang berulang kali gagal login
- 6** **Rutin jalankan pembaruan keamanan**
`sudo apt update && sudo apt upgrade -y`

Hardening Tidak Dikerjakan Manual

Enam langkah tadi ditulis sebagai KODE,
bukan dikerjakan satu per satu di tiap server.

- Di perusahaan, hardening ditulis sebagai playbook Ansible atau skrip cloud-init yang jalan otomatis setiap kali server baru dibuat
- Server yang dibuat hari ini dan tahun depan akan diamankan dengan cara yang persis sama — tidak ada langkah yang terlupa
- Inilah pekerjaan inti DevOps: mengubah daftar langkah manual menjadi kode yang bisa diulang
- Kalau kalian bisa menjelaskan keenam langkah ini beserta alasannya saat wawancara magang, kalian sudah unggul dari kebanyakan pelamar seusia kalian

3

SESI 3 · 35 MENIT

Virtualisasi

Kenapa satu server dibelah jadi banyak, apa itu hypervisor, dan mengapa container lahir sebagai penerusnya.

Era 1990-an: Satu Server, Satu Aplikasi

Punya 5 aplikasi berarti beli 5 server fisik — karena aplikasi sering saling bentrok.

5–15%

Rata-rata pemakaian server saat itu.
Sisanya menganggur

100%

Listrik, ruang, pendingin, dan perawatan
tetap dibayar penuh

60–80%

Pemakaian setelah virtualisasi
diterapkan

Sebelum memberi jawaban, tanyakan dulu ke kelas: "Kalian punya 5 komputer tapi masing-masing cuma dipakai 10%. Idenya apa?" — Biarkan mereka sendiri yang sampai ke gagasan menggabungkan. Mereka baru saja menemukan virtualisasi.

Tipe 1 — Bare Metal

Dipasang langsung di atas perangkat keras, tanpa sistem operasi di bawahnya.

```
+-----+-----+-----+
| VM 1 | VM 2 | VM 3 |
| Ubuntu | Windows | CentOS |
+-----+-----+-----+
|      HYPERVISOR      | <- Proxmox, VMware ESXi,
+-----+-----+      Hyper-V, KVM, XenServer
| PERANGKAT KERAS FISIK |
+-----+-----+
```

Cepat, stabil, dipakai di pusat data dan oleh SEMUA penyedia cloud. VPS yang kalian sewa berjalan di atas hypervisor Tipe 1.

Tipe 2 — Hosted

Berjalan sebagai aplikasi biasa di atas sistem operasi yang sudah ada.

```
+-----+-----+
| VM 1 | VM 2 |
+-----+-----+
|  HYPERVISOR  | <- VirtualBox,
+-----+-----+   VMware Workstation, Parallels
| OS HOST (Win/Mac)|
+-----+-----+
| PERANGKAT KERAS |
+-----+-----+
```

Lebih lambat karena harus melewati OS host, tapi mudah dipasang. Biasa dipakai di laptop dan komputer lab untuk belajar.

Type 1 vs Type 2

Aspek	Tipe 1 (Bare Metal)	Tipe 2 (Hosted)
Letak	Langsung di perangkat keras	Di atas OS host
Kinerja	Tinggi	Sedang
Pemasangan	Rumit	Mudah
Pemakaian	Pusat data, produksi	Belajar, uji coba, pengembangan
Contoh	Proxmox, ESXi, Hyper-V, KVM	VirtualBox, VMware Player

Cek paham: VirtualBox di laptop kalian tipe berapa? Server di Biznet atau AWS tipe berapa?

Kelebihan & Keterbatasan VM

Kelebihan

Kenapa VM masih dipakai luas

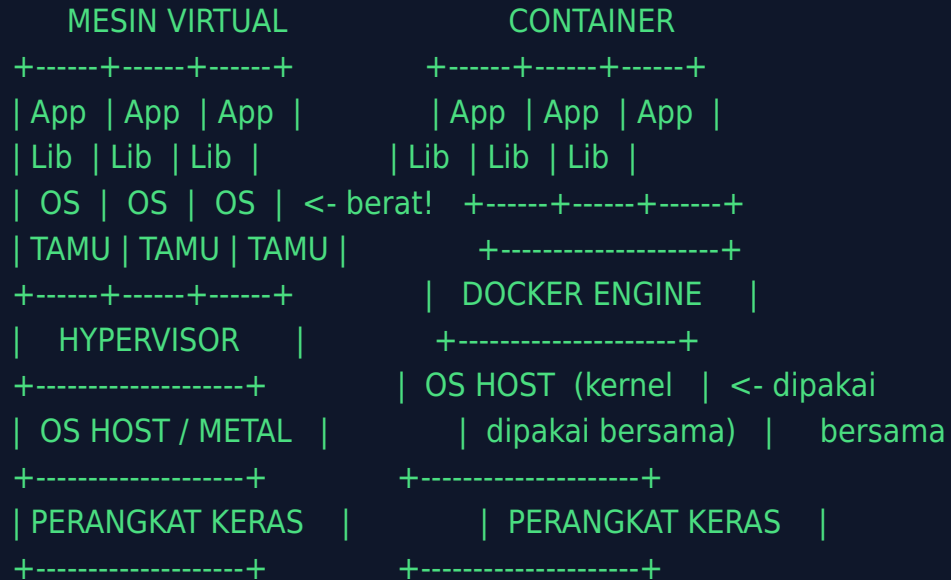
- Satu perangkat keras menjalankan banyak OS berbeda
- Isolasi sangat kuat — satu VM jatuh tidak menyeret yang lain
- Bisa di-snapshot untuk dikembalikan ke kondisi sebelumnya
- Bisa dipindah antar server fisik
- Pembagian sumber daya bisa diatur dengan tegas

Keterbatasan

Semuanya berakar pada satu hal

- Setiap VM membawa sistem operasi lengkapnya sendiri
- Ukuran image besar — satuan gigabyte
- Waktu menyala lama — puluhan detik sampai menit
- RAM banyak terpakai hanya untuk OS yang menganggur
- Setiap OS tamu harus di-update & diamankan satu per satu

Kenapa Harus Ada 5 Salinan Linux?



Tiap kotak = 1-20 GB
Menyala = puluhan detik

Tiap kotak = 5-200 MB
Menyala = di bawah 1 detik

Container menjawab: pakai bersama kernel milik OS host. Yang dikemas hanya aplikasi dan pustakanya — bukan seluruh sistem operasi.

Mesin Virtual vs Container

Aspek	Mesin Virtual	Container
Isi paket	OS lengkap + aplikasi	Aplikasi + pustakanya saja
Ukuran khas	1-20 GB	5-200 MB
Waktu menyala	30 detik - beberapa menit	Di bawah 1 detik
Kernel	Punya sendiri	Dipakai bersama host
Tingkat isolasi	Sangat kuat (perangkat keras)	Cukup kuat (lapisan proses)
Kepadatan per server	Puluhan	Ratusan hingga ribuan
OS berbeda di satu host	Bisa (Windows di atas Linux)	Tidak bisa

ANALOGI

VM itu rumah tapak.
Container itu unit apartemen.

Rumah tapak: tiap rumah punya pondasi, dinding, atap, listrik, dan saluran air sendiri.
Sangat mandiri — tapi butuh lahan luas dan lama membangunnya.

Apartemen: pondasi, atap, dan saluran utama dipakai bersama. Tiap unit hanya punya sekat dan perabot sendiri.
Jauh lebih hemat lahan dan cepat dibangun.

Dari analogi ini, apa kelemahan container? — Isolasinya lebih tipis daripada VM.

Kapan Pakai yang Mana?

Kenyataannya di lapangan: keduanya dipakai bersamaan.

Pakai VM ketika...

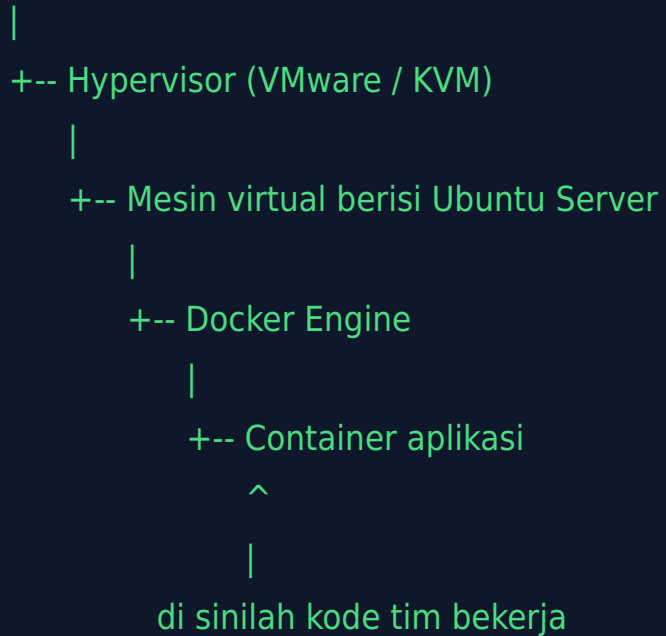
- Butuh OS yang berbeda dari host (Windows di atas Linux)
- Butuh isolasi setingkat keamanan tinggi
- Menjalankan aplikasi lama yang menuntut versi kernel tertentu

Pakai container ketika...

- Menjalankan banyak layanan kecil
- Butuh deploy cepat dan sering
- Ingin lingkungan pengembangan dan produksi yang identik
- Ingin memadatkan sebanyak mungkin layanan di satu server

Susunan Nyata di Perusahaan

Perangkat keras di data center



Seorang DevOps engineer bekerja di dua lapisan terbawah: menyiapkan VM-nya, lalu mengatur container yang jalan di atasnya. Itu sebabnya kalian perlu paham keduanya.

4

SESI 4 · 55 MENIT

Pengenalan Docker + Praktik

Masalah yang dipecahkannya, arsitektur dan empat konsep intinya — lalu kalian menjalankan container pertama sendiri di VPS masing-masing.

KELUHAN PALING TERKENAL DI DUNIA PERANGKAT LUNAK

"Di laptop saya jalan, kok."

Penyebabnya selalu berkisar di hal yang sama: versi PHP berbeda, pustaka kurang, variabel lingkungan belum diatur, sistem operasinya lain, atau ada berkas konfigurasi yang lupa disalin.

Gagasan Docker: jangan hanya kirim kodenya.

Kirim kode BERSAMA seluruh lingkungan tempat kode itu terbukti jalan.

ASAL-USUL NAMANYA

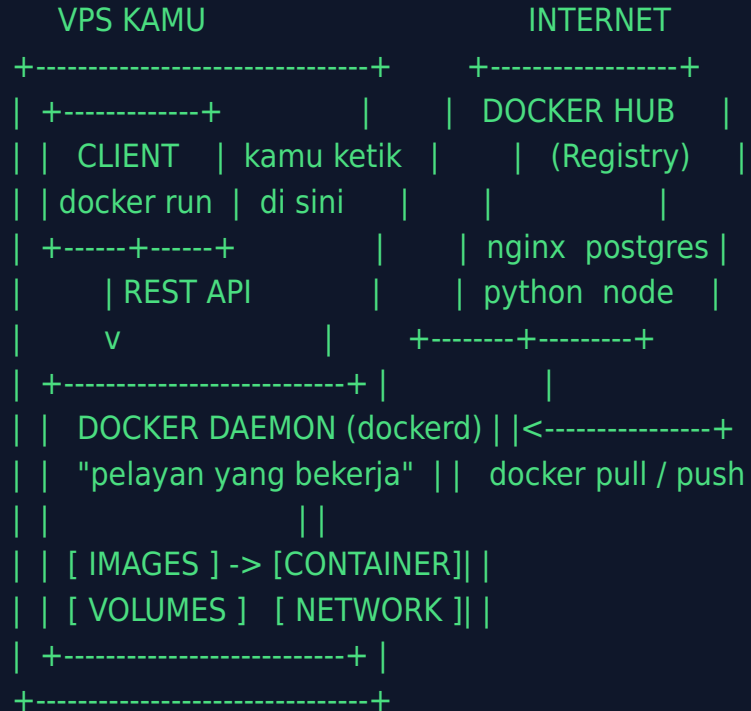
Isinya boleh apa saja.
Ukuran luarnya selalu sama.

Sebelum 1950-an, barang dimuat ke kapal satu per satu: karung, peti kayu, drum. Lama, mahal, sering rusak.

Lalu ditemukan kontainer baja berukuran standar. Isinya boleh pisang, mobil, elektronik — tapi ukuran luarnya sama. Akibatnya satu derek, satu kapal, satu truk yang sama bisa mengangkut semuanya.

Docker melakukan hal yang persis sama untuk perangkat lunak: isinya boleh Laravel, Node.js, atau Python — perintah menjalankannya tetap sama.

Tiga Bagian Utama Docker



Client hanya menyampaikan permintaan. Daemon yang mengerjakan semuanya. Kalau muncul "Cannot connect to the Docker daemon", artinya client hidup tapi daemon mati — di VPS: `sudo systemctl start docker`.

Empat Konsep yang Harus Diingat

Kalau siswa hanya ingat empat hal dari dua hari ini, harus yang ini.

1

IMAGE

Paket siap pakai, hanya-baca. Berisi aplikasi + semua yang dibutuhkan.

Cetakannya.

2

CONTAINER

Image yang sedang dijalankan. Punya lapisan tulis sendiri.

Hasil jadinya.

3

VOLUME

Ruang penyimpanan yang hidupnya terpisah dari container.

Agar data tidak hilang.

4

NETWORK

Jaringan virtual tempat container saling memanggil dengan nama.

Jalan komunikasinya.

Image = Resep · Container = Kue

Di Docker	Di dapur	Di pemrograman
Image	Resep + bahan yang sudah ditakar	Class
Container	Kue yang sudah jadi	Object
1 image → banyak container	1 resep → banyak kue	1 class → banyak object
Image tidak berubah	Resep tidak berubah saat dipakai	Class tidak berubah
Container bisa dihapus	Kue bisa dimakan atau dibuang	Object bisa dihapus

Pertanyaan penguji: "Kalau kuenya saya makan, resepnya ikut hilang tidak?" — Tidak. Itulah kenapa container bisa dihapus dan dibuat ulang kapan saja tanpa risiko.

Kenapa Data Tidak Boleh Ditaruh di Dalam Container

- Container bisa dihapus kapan saja — dan lapisan tulisnya ikut terhapus
- Volume adalah ruang penyimpanan yang hidupnya terpisah dari container
- Container dihapus → volume tetap ada
- Container baru bisa dipasangkan ke volume lama → datanya kembali
- Wajib untuk database, unggahan pengguna, dan berkas konfigurasi

Analogi

Container = kamar kos

Volume = loker penyimpanan
di luar kamar

Kamu boleh pindah kamar berkali-kali; isi loker tidak ikut terbuang.

Kalau baju disimpan di dalam kamar dan kamarnya dibongkar, bajunya ikut hilang.

Named Volume vs Bind Mount

Aspek	Named Volume	Bind Mount
Ditulis	-v datasaya:/var/lib/data	-v ./folder:/app
Dikelola oleh	Docker	Kamu sendiri
Letaknya	Di area internal Docker	Folder di host (VPS-mu)
Cocok untuk	Data produksi, database	Kode saat sedang dikembangkan
Lintas sistem operasi	Aman	Perlu perhatian ekstra di Windows

Besok kita pakai named volume untuk latihan penyimpanan: dikelola Docker sepenuhnya, jadi tidak bergantung pada letak folder di VPS.

Pemetaan Port — Wajib Dihafal

```
docker run -p 8080:80 nginx
```

```
| |
```

```
| +--- port DI DALAM container
```

```
| (port yang didengarkan nginx)
```

```
|
```

```
+----- port DI HOST
```

```
(mesin tempat Docker berjalan)
```

Di host itu: <http://localhost:8080>

diteruskan ke port 80 di container

"Luar dulu, baru dalam": angka kiri dunia luar (host), angka kanan dunia dalam (container). Di lab kita, host-nya adalah VPS — cara membukanya dari browser PC ada di praktik sebentar lagi.

Resep untuk Membuat Image Sendiri

- FROM — image dasar, selalu baris pertama
- WORKDIR — folder kerja di dalam image
- COPY — salin berkas dari host ke image
- RUN — jalankan perintah saat membangun image
- EXPOSE — catat port yang dipakai (hanya dokumentasi)
- ENV — tetapkan variabel lingkungan
- CMD — perintah default saat container dinyalakan

Contoh lengkap

```
FROM nginx:alpine
```

```
LABEL maintainer="siswa@smk.id"
```

```
RUN rm -rf /usr/share/nginx/html/*
```

```
COPY site/ /usr/share/nginx/html/
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

RUN vs CMD

RUN → dijalankan SATU KALI saat image dibangun

CMD → dijalankan SETIAP KALI container dinyalakan

- RUN menghasilkan lapisan yang ikut tersimpan ke dalam image. Contoh: memasang paket
- CMD tidak tersimpan sebagai lapisan — ia hanya instruksi "kalau nanti dijalankan, jalankan ini"
- Analoginya: RUN = menyiapkan bahan waktu memasak. CMD = menghidangkan ke meja
- Membangun image: `docker build -t namaimage:1.0 .` ← titik berarti "bahan bangunannya ada di folder ini"

Menjalankan Banyak Container Sekaligus

- Aplikasi nyata jarang hanya satu container
- Contoh: satu container untuk web, satu lagi untuk database
- Semua ditulis di satu berkas docker-compose.yml
- `docker compose up -d` → nyalakan semua sekaligus
- Nama service (db) langsung jadi nama host di jaringan Docker
- YAML pakai SPASI, jangan Tab — satu Tab membuat berkas ditolak

docker-compose.yml

```
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_PASSWORD: rahasia
    volumes:
      - data-db:/var/lib/postgresql/data

volumes:
  data-db:
```

Praktik: Container Pertama Milikmu

- 1** Pastikan Docker di VPS-mu hidup
`docker version` `docker run hello-world`
- 2** Jalankan web server di port 8080
`docker run -d -p 8080:80 --name web-pertama nginx:alpine`
- 3** Buktikan dari sisi VPS
`docker ps` `curl localhost:8080`
- 4** Buka dari browser PC → tunnel SSH, di PowerShell/CMD BARU
`ssh -N -L 8080:localhost:8080 siswa@IP-VPS`
- 5** Buktikan, lalu bersihkan
`docker logs web-pertama` `docker stop web-pertama` `docker rm web-pertama`

Tunnel SSH: Menyambung Pipa lewat Port 22

Dari PC, VPS hanya bisa dihubungi lewat port 22. Jadi kita menyambung "pipa" di dalam jalur SSH itu.

BROWSER PC	PC (Windows)	VPS	CONTAINER
localhost:8080 -->	pintu 8080 =====	SSH port 22 =====>	localhost:8080 --> nginx :80
	dibuka oleh	dibuka oleh	
	ssh -L 8080:localhost:8080	docker run -p 8080:80	

```
ssh -N -L 8080:localhost:8080 siswa@IP-VPS
```

```
| | | |
| | | +-- port tujuan di VPS
| | +----- "localhost" dilihat dari VPS
| +----- port yang dibuka di PC
+----- -N: hanya pipa, tanpa jendela perintah
```

Jendela tunnel terlihat diam — itu normal, pipanya aktif selama jendela terbuka. Buka <http://localhost:8080> di browser PC.

Kalau Gagal, Baca Pesannya

Pesan / gejala	Penanganan
Permission denied, please try again.	Password atau akun salah. Password memang tidak terlihat saat diketik
docker: command not found	Kamu masih di PC Windows. Lihat awal baris, masuk SSH dulu
Cannot connect to the Docker daemon	Layanan docker mati: <code>sudo systemctl start docker</code>
permission denied ... docker.sock	exit, lalu masuk SSH lagi. Kalau tetap, panggil asisten
port is already allocated	Container lamamu masih memakai 8080: <code>docker ps</code> , lalu <code>docker rm -f</code>
bind ... 8080: Address already in use	Tunnel lama masih terbuka — tutup, atau pakai <code>-L 18080:localhost:8080</code>
Browser: This site can't be reached	Jendela tunnel tertutup. Jalankan lagi <code>ssh -N -L ...</code>

Tambahkan ke daftar "Yang merepotkan hari ini": dua jendela, password berkali-kali, perintah tunnel yang panjang.



4.7

SESI 4.7 · 4 MENIT

Docker di Dunia Kerja

Bagian singkat yang membuat siswa paham: ini bukan latihan sekolah, ini keterampilan yang dibayar.

Kenapa DevOps Lahir

Dulu ada dua kubu yang sering bertengkar.

Tim Developer

Maunya fitur baru cepat naik

- "Kode saya sudah benar, servernya yang aneh"
- Ingin deploy sesering mungkin
- Tidak menguasai seluk-beluk server

Tim Operations

Maunya server stabil

- "Tiap kali kalian deploy, server saya bermasalah"
- Ingin perubahan sesedikit mungkin
- Tidak menguasai isi kode aplikasinya

Kolom Kiri Isinya Lowongan Kerja

Pekerjaan sehari-hari	Materi kita yang dipakai
Menyiapkan server baru untuk aplikasi	Cloud, VPS, SSH, firewall (Sesi 1-2)
Membungkus aplikasi jadi image	Dockerfile (Lab 3 besok)
Menjalankan aplikasi + database bersamaan	Docker Compose (Lab 4 besok)
Memastikan data tidak hilang saat deploy	Volume (Lab 2 besok)
Mencari tahu kenapa aplikasi mati jam 2 pagi	docker logs, journalctl (Sesi 2)
Membuat proses deploy berjalan otomatis	CI/CD — lanjutan dari semua ini

Ucapkan apa adanya: "Kolom kanan isinya materi dua hari ini. Kolom kiri isinya lowongan kerja. Kalian sedang belajar hal yang sama."

Tidak Ada yang Langsung Lompat ke Puncak

Siswa SMK TKJ

|

+-- Teknisi jaringan / IT support

|

+-- System administrator <- Sesi 1 & 2 hari ini

|

+-- DevOps engineer <- Sesi 3 & 4 + praktek besok

|

+-- Site Reliability Engineer

Cloud Architect

Yang membedakan orang yang naik cepat dan yang berhenti di tengah biasanya bukan bakat — melainkan kebiasaan mendokumentasikan, memverifikasi, dan tidak asal mengaku sudah beres.

Lima Kebiasaan yang Dinilai di Tempat Kerja

- 1 Tulis semuanya sebagai berkas, bukan ingatan**
Dockerfile, compose, skrip. Kalau hanya ada di kepala, ia akan hilang.
- 2 Verifikasi dengan bukti, bukan perasaan**
Buka browsernya. Jalankan docker ps. Baca lognya.
- 3 Baca pesan error sampai selesai**
Jawabannya biasanya sudah tertulis di sana.
- 4 Jangan simpan sandi di dalam kode**
Pakai berkas .env yang tidak ikut di-commit ke Git.
- 5 Anggap server bisa mati kapan saja**
Karena itu data disimpan di volume, dan cara membangun ulang server ditulis sebagai kode.



PENUTUP · 10 MENIT

Recap & Tugas untuk Besok

Sepuluh poin kunci, kuis cepat, dan persiapan untuk hari kedua.

Sepuluh Poin yang Dibawa Pulang

- 1 Server adalah komputer yang melayani dan menyala terus
- 2 Cloud = menyewa • On-premise = memiliki. Tergantung pola pemakaian, bukan tren
- 3 IaaS / PaaS / SaaS — makin ke kanan makin sedikit yang diurus & yang bisa diatur
- 4 VPS adalah bentuk IaaS paling terjangkau untuk belajar
- 5 SSH adalah pintu masuk server; kunci SSH lebih aman daripada password
- 6 Izinkan port 22 SEBELUM menyalakan firewall — atau terkunci di luar
- 7 Virtualisasi memecah satu server fisik jadi banyak VM; hypervisor yang mengatur
- 8 Container lebih ringan dari VM karena memakai kernel host bersama-sama
- 9 Image = cetakan, container = hasil jadinya. Satu image, banyak container
- 10 Data penting disimpan di volume, bukan di dalam container

Tunjuk Acak, Jawab Singkat



Soal 1 – 4

1. Gmail itu IaaS, PaaS, atau SaaS?
2. VirtualBox hypervisor tipe berapa?
3. Mana lebih cepat menyala, VM atau container?
4. `-p 9090:80` — browser dibuka di port berapa?



Soal 5 – 7

5. Image itu cetakan atau hasil jadi?
6. Container dihapus, data di volume ikut hilang?
7. Port SSH berapa?



Soal 8 – 10

8. Perintah melihat container yang sedang berjalan?
9. Kenapa tidak boleh bekerja sebagai root?
10. Apa yang harus diizinkan sebelum menyalakan firewall?

Tugas untuk Besok

Simpan kartu akunmu — besok VPS-nya sama.

- Berkas dan image kalian hari ini masih ada di VPS besok pagi
- Besok kita memakai PC lab yang sama — tidak ada yang perlu dipasang di rumah
- Ingat bagian yang paling merepotkan hari ini: password berulang, dua jendela, perintah tunnel yang panjang
- Besok pagi kita mulai dari daftar keluhan itu
- Opsional: baca bagian 1.4 dan 3.5 di modul yang tadi dipangkas

SAMPAI JUMPA BESOK

Hari ini kalian merasakan repotnya.
Besok kalian mendapat alatnya.

Hari 2: empat lab terbimbing dengan alat baru, lalu ujian teori dan ujian praktek (kembali manual).

Yang kalian bangun besok — sebuah aplikasi web dan database yang berjalan bersama dari satu berkas — adalah hal yang dikerjakan DevOps engineer setiap hari di tempat kerja.